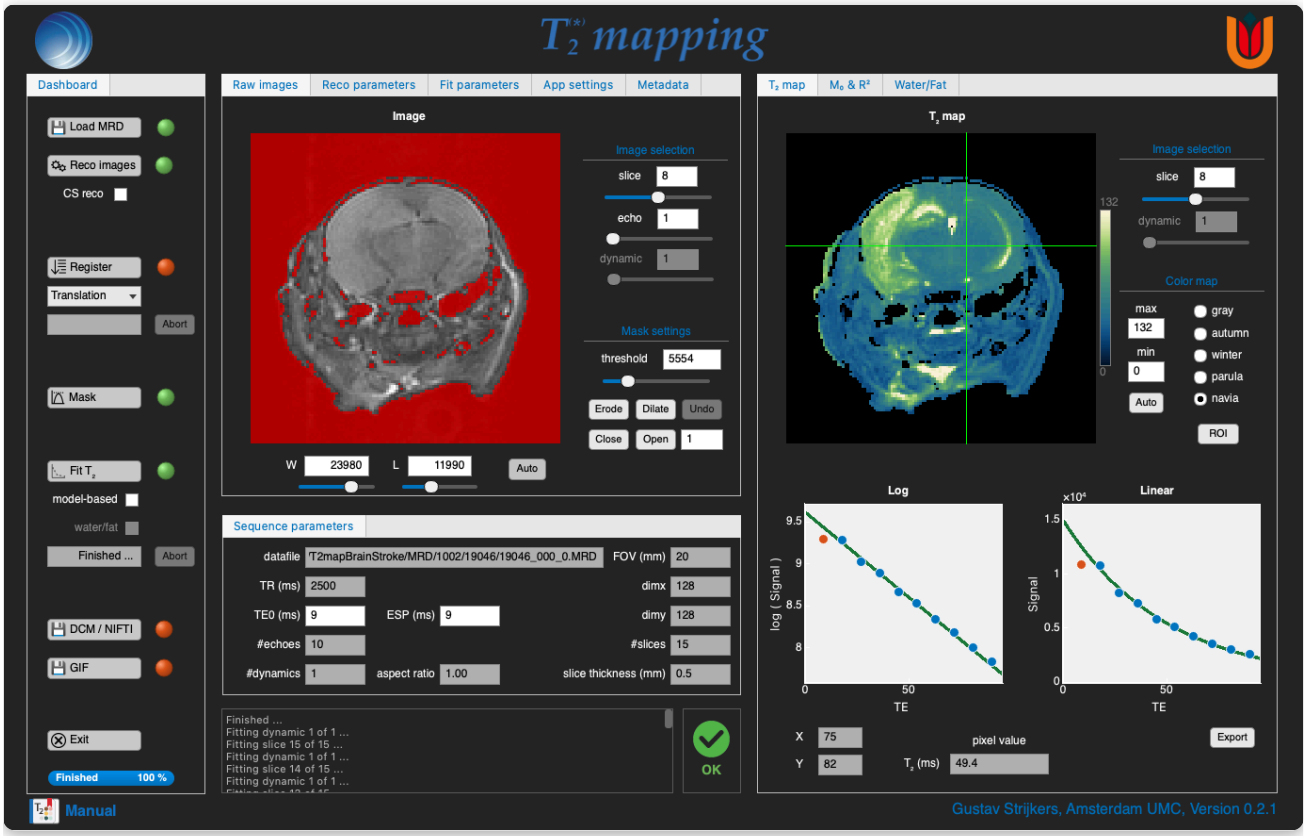


Table of contents

1. Overview
2. Contact
3. Disclaimer
4. Installation
5. Running the software
6. Basic operation
7. Credits

1. Overview

T2mapp is an application with a graphical user interface to reconstruct T_2 maps from multi-echo spin echo, and T_2^* maps from multi-gradient echo experiments acquired with an MR Solutions preclinical MRI system. The images are reconstructed, registered, masked and fitted to T_2 (or T_2^*), M_0 and R^2 maps; multi-gradient echo data can also be separated into water and fat maps. The maps can be exported as DICOM, NIfTI, BIDS and GIF, with the values of a region of interest as a CSV file.

T2mapp is a stand-alone application: everything it uses, elastix and dcm2niix included, is inside it, and nothing else has to be installed.

2. Contact

T2mapp and this manual are written by:

Gustav Strijkers
Amsterdam UMC
Biomedical Engineering and Physics
Amsterdam, the Netherlands
g.j.strijkers@amsterdamumc.nl

3. Disclaimer

This is a beta version. The software has been extensively tested using mouse and rat data acquired with an MR Solutions preclinical 7.0T/24cm system. However, this does not warrant that the functions contained in the program will meet your requirements or that the operation of the program will be uninterrupted or error-free. In case of questions or issues, please contact Gustav Strijkers.

4. Installation

macOS

Requirements: a Mac with Apple silicon (M1 or later), macOS 26 or later.

T2mapp comes as a disk image, `T2mapp-0.3.1-macOS-arm64.dmg`. Nothing else has to be installed: Python, elastix, dcm2niix and everything else the app uses are inside it.

1. Open the disk image and drag **T2mapp** onto the Applications folder in its window.
2. Open it from Applications. The app is not signed with an Apple Developer ID, so the first time macOS refuses it ("Apple could not verify ..."); click **Done**. Then open **System Settings > Privacy & Security**, scroll down to the line saying T2mapp was blocked, click **Open Anyway** and confirm. From then on it opens as any other app.

   Instead of step 2, the same in Terminal, once:

   ```
   xattr -dr com.apple.quarantine /Applications/T2mapp.app
   ```

The first start can take a little longer: macOS checks the app once.

Windows

Requirements: a PC with Windows 10 or 11, 64-bit.

T2mapp comes as an installer, `T2mapp-0.3.1-setup.exe`. Nothing else has to be installed: Python, elastix, dcm2niix and everything else the app uses are inside it.

1. Run the installer. It is not signed, so the first time Windows may stop it ("Windows protected your PC"): click **More info** and then **Run anyway**.
2. The installer puts the app in `%LOCALAPPDATA%\Programs\T2mapp` for the current user, without administrator rights (installing it for all users is an option), with a Start-menu entry and, if you choose, a desktop shortcut. At the end it can start the app.

To remove the app, use **Uninstall T2mapp** in the Start menu, or **Settings > Apps**.

BART (optional)

With BART, the Berkeley Advanced Reconstruction Toolbox (https://mrirecon.github.io/bart/), version 1.0.00 or newer, the compressed sensing reconstruction runs with BART, and the model-based T_2 fit is available. Without BART, or with an older version, the compressed sensing reconstruction runs with the app's own engine, based on sigpy, and the model-based fit is switched off. The message box says so at start. When BART is built with GPU support, T2mapp runs it on the GPU.

Installing BART on macOS with MacPorts

1. Install the Xcode command line tools:

   ```
   xcode-select --install
   ```

2. Install MacPorts from https://www.macports.org/.

3. Install the packages:

```
sudo port install fftw-3-single gcc14 libpng openblas flock gmake coreutils l
ibblocksruntime
```

4. Download and unpack BART:

```
wget https://github.com/mrirecon/bart/archive/v1.0.00.tar.gz
tar xvfz v1.0.00.tar.gz
cd bart-1.0.00
```

5. Build and install:

```
export PATH=/opt/local/libexec/gnubin:$PATH
CC=gcc-mp-14 gmake
sudo gmake install
```

   The compiler name must match the gcc version installed in step 3: gcc14 gives gcc-mp-14, gcc15 gives gcc-mp-15.

6. Check that it works: `bart version`

Installing BART on macOS with Homebrew

1. Install the Xcode command line tools: `xcode-select --install`
2. Install Homebrew:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/instal
l/HEAD/install.sh)"
```

3. Install the packages BART needs:

```
brew install gcc libpng fftw openblas make llvm libomp
```

4. Download and unpack BART as above (step 4 of MacPorts).
5. Build and install:

```
CC=gcc-15 MACPORTS=0 gmake
sudo gmake install
```

   The compiler name must match the version Homebrew installed; check it with `ls /opt/homebrew/bin/gcc-*`.

6. Check that it works: `bart version`

How T2mapp finds BART on macOS

`sudo make install` puts BART in `/usr/local/bin`, which is where the app looks first. It looks in these folders, in this order:

Folder	
`/usr/local/bin`	where BART's own `make install` puts it
`/usr/bin`	
`/opt/homebrew/bin`	Homebrew's
`/opt/local/bin`	MacPorts'

An app opened from the Finder or the Dock does not see the settings of your shell (such as `TOOLBOX_PATH` or `PATH` in your shell profile). If your `bart` is in another folder, a link to it in `/usr/local/bin` lets the app find it. In Terminal, once (Terminal asks for your password):

```
sudo mkdir -p /usr/local/bin
sudo ln -s /path/to/bart/bart /usr/local/bin/bart
```

Installing BART on Windows 11

On Windows, BART runs inside the Windows Subsystem for Linux (WSL); the app calls it as `wsl bart`.

1. Install WSL. Start a Windows PowerShell as administrator and run `wsl --install`. This installs WSL 2 and Ubuntu; a restart is needed. The first time you start the Linux prompt you are asked to create a user account. See https://pureinfotech.com/install-wsl-windows-11/ for older systems.
2. Update the Linux distribution:

   ```
   sudo apt-get update
   sudo apt-get dist-upgrade
   ```

3. Install the packages BART needs (gcc 12 or newer):

   ```
   sudo apt-get install make gcc libfftw3-dev liblapacke-dev libpng-dev libopenb
   las-dev gfortran
   ```

4. Download, build and install BART:

   ```
   wget https://github.com/mrirecon/bart/archive/v1.0.00.tar.gz
   tar xvfz v1.0.00.tar.gz
   cd bart-1.0.00
   make
   sudo make install
   ```

5. Check from a Windows command prompt: `wsl bart version`

Elastix registration

Image registration uses elastix, which comes with T2mapp. When elastix cannot be run, T2mapp registers the images with a registration of its own instead (a B-spline registration is then done as affine), and says so.

5. Running the software

Open **T2mapp** from the Applications folder (macOS), or from the Start menu or the desktop shortcut (Windows). The T2mapp logo appears while the app loads, and the window follows within seconds. At its start the app looks for elastix and BART, and the message box reports what it found; "Ready" says it is done. Only **Load MRD** and **Exit** are on until a scan is loaded.

Newer versions. At its start the app asks GitHub (https://github.com/Moby1971/MRI-apps) whether there is a newer release of it. The message box then shows the current and the newest version and, when there is a newer one, where to download it. Without a network connection it says that the version could not be checked, and the app works as usual. `checkForUpdates` in the settings file switches the check off; `versionCheckTimeout` sets how many seconds it waits for GitHub (5).

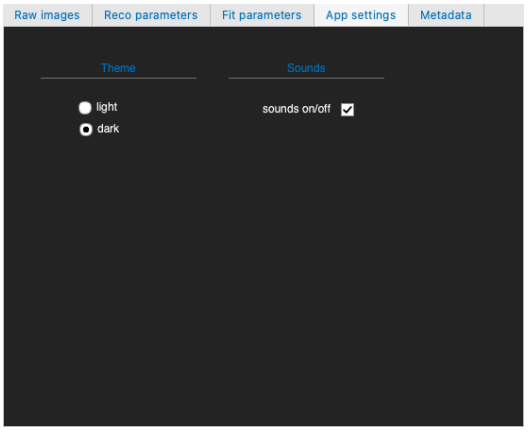
This manual. The **Manual** button at the bottom of the window, with the app's icon on a book, opens this manual in the computer's PDF viewer.

The window has a fixed size. On a screen too small for it, the window fits the screen and can be scrolled.

Settings and log files

T2mapp keeps its files in a folder of its own, which it makes at its first start:

macOS	`~/Library/Application Support/T2mapp/0.3.1p`
Windows	`%APPDATA%\T2mapp\0.3.1p`



Settings. The settings are kept in `t2mappSettings.json`. The file is written with the standard values the first time a version starts, and can then be edited by hand; changes take effect at the next start. It holds the version check, the folders where loading and the exports start (`mrdImportPath`, `defaultExportPath`), the theme, the sounds, the values the reconstruction, registration and fit fields start with, the color maps of the maps, and the number of log files kept. A value that cannot be used is replaced by its standard value, and the message box says so; a file that cannot be read at all is set aside as `t2mappSettings-unreadable-<time>.json` and replaced by a new one. The theme and the sounds can also be switched in the App settings tab, for the session only.

Log files. Every session writes a log file, `t2mapp-log-<date and time>.txt`, to the same folder. The log keeps every message and the parameters of each import, reconstruction, registration and fit. Logs older than the newest 50 are removed at start.

6. Basic operation



The T2mapp program operates from a single window with 5 panels.

Panel 1: Dashboard. The task buttons and parameters that control the reconstruction process. The tasks run from top to bottom; registration and masking can be left out. A green light shows a task done, yellow a task running, and red one not done or failed. At the bottom of the Dashboard one progress bar shows the progress of every task ("Registration 40 %", "Fit 40 %", "Finished").

Panel 2: Raw images / Reco parameters / Fit parameters / App settings / Metadata. The **Metadata** tab holds the record of the scan: the animal, the session and the contrast agent (step 9).

Panel 3: Sequence parameters. The relevant acquisition parameters. The first echo time (**TE0**) and the echo spacing (**ESP**) can be edited; changing them after a fit clears the maps.

Panel 4: T_2 map / M_0 & R^2 / Water/Fat. During and after fitting, the T_2 (or T_2*) maps. The M_0 and R^2 goodness-of-fit maps are displayed in the second tab, the water and fat maps in the third.

Panel 5: Messages. The program messages, newest on top, and the status: OK, BUSY, WARNING or ERROR.

The mouse scroll wheel steps through the slices: over the raw images those of the images, over the maps those of the maps.

Step 1: Loading data

Press **Load MRD** to load the MRD raw data file. On the scanner computer (Windows) the first dialog of a session opens in the newest exam of the scanner's MRD folder, unless the settings name a folder (`mrdImportPath`).

T2mapp recognizes multi-echo spin echo and multi-gradient echo scans from the MRD parameters; a single-echo scan is refused, with the reason in the message box. Undersampled (compressed sensing) scans are sorted with their trajectory table, the `.txt` file beside the MRD file. Retrospective's and P2ROUD's MRD exports load as well.

For multi-receiver coil data, select one of the MRD files; the app loads the others. The relevant acquisition parameters are shown in panel 3.

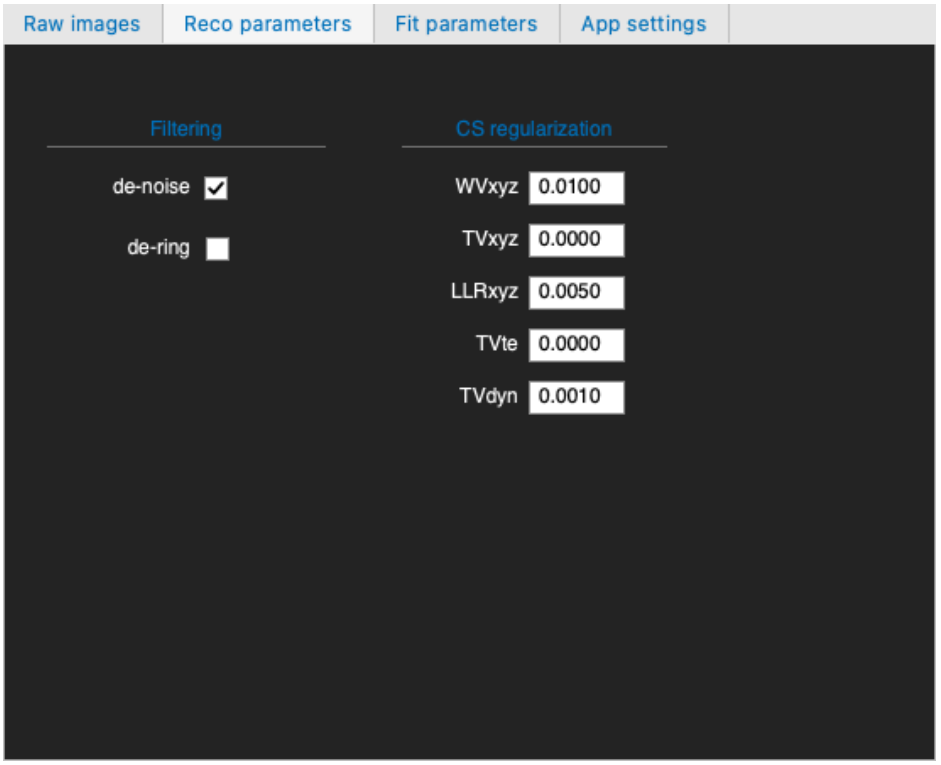


Echo times. TE0 and **ESP** are filled in when the scan is loaded. When the message box asks you to, check them before fitting.

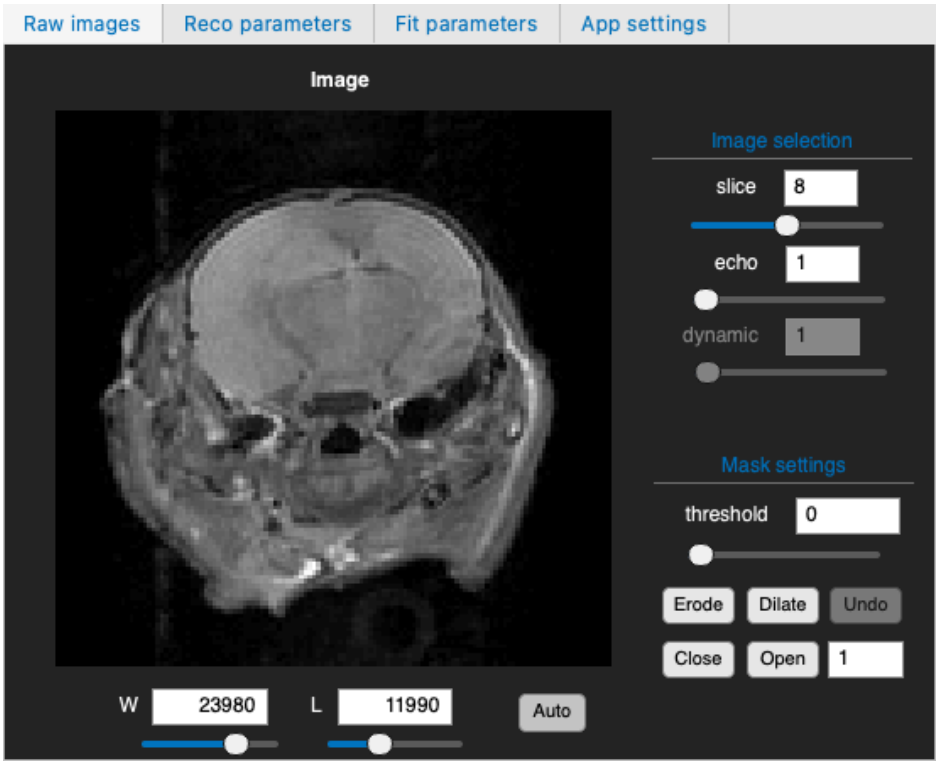
Step 2: Image reconstruction

Loading reconstructs the images already. Press **Reco images** to reconstruct them again, for instance with compressed sensing.

The standard reconstruction is a 2D or 3D FFT with PCA de-noising, on by default. When the **CS reco** checkbox is checked, the images are reconstructed by compressed sensing with the weights of the Reco parameters tab: wavelets (WV), total variation (TV) and locally low rank (LLR) in space (xyz), and total variation along the echoes (TVte) and the dynamics (TVdyn). That tab also switches PCA de-noising (**de-noise**) and Gibbs ringing suppression (**de-ring**, off by default), which removes the ripple beside sharp edges.



Images with different echo times and dynamics can be inspected with the sliders of the Raw images tab (**slice**, **echo** and **dynamic**). The window and level are set with the **W** and **L** fields and sliders, or with **Auto**.



Step 3: Image registration

NOTE: this step usually can be omitted.

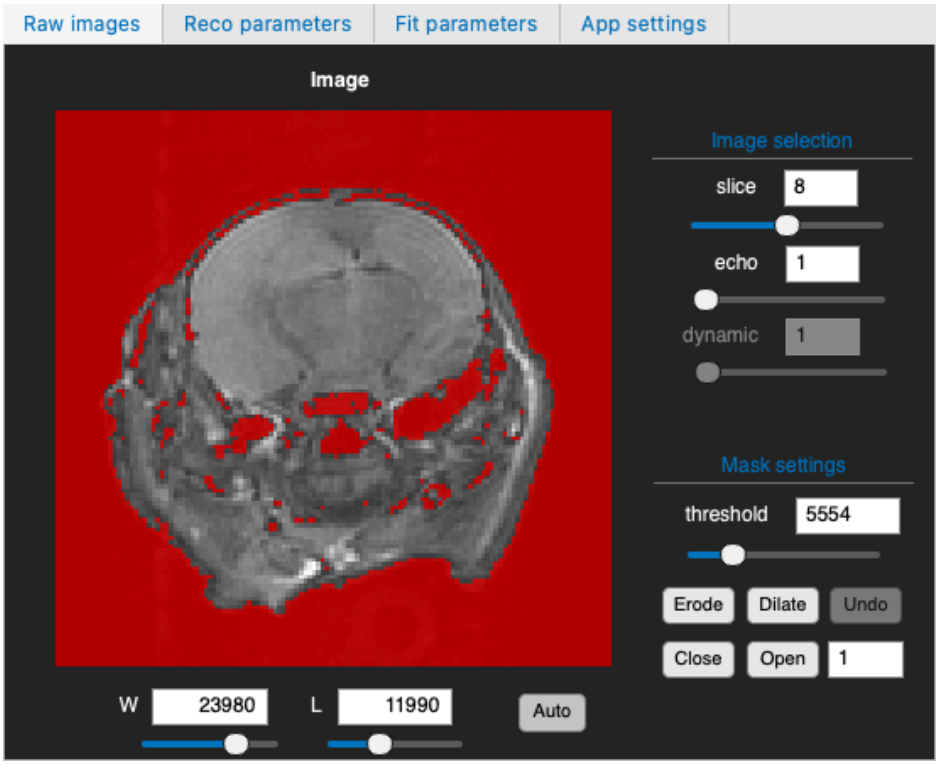
In case the images of the different echoes are not perfectly registered, press **Register** to register every echo to the first echo of its slice and dynamic with elastix, as a **Translation**, **Rigid**, **Affine** or **B-Spline** transform (the list below the button). **Abort** stops it; the images are then partly registered. Registering clears the mask and the maps.

Step 4: Masking the background

To prevent unnecessary fitting of the background pixels, the images need to be masked first; without a mask every pixel is fitted, the background included.

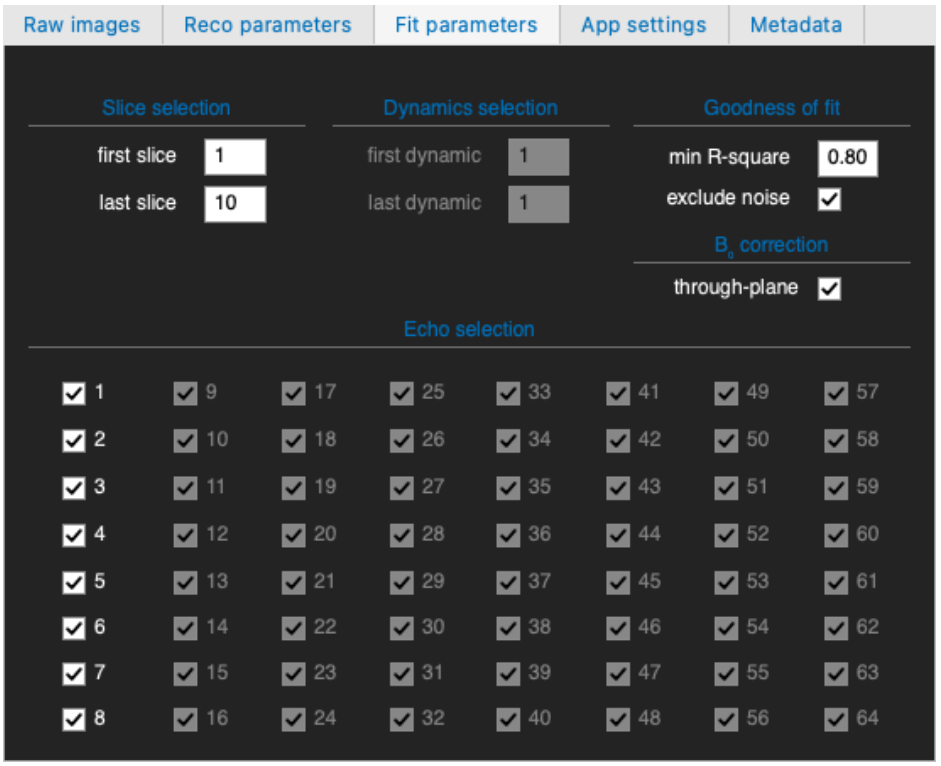
Press **Mask** to mask them with an automatic (Otsu) threshold on the first echo, per slice. The red pixels indicate the regions that will be skipped during T_2 fitting.

In case the automatic threshold is not satisfying, it can be adjusted with the **threshold** field or slider, for all slices. The mask can be refined by **Erode**, **Dilate**, **Close** and **Open** operations, of the size in pixels in the field beside them; **Undo** takes the last one back.



Step 5: Fitting T_2 and T_2* maps

The Fit parameters tab holds the fit settings.



The first and last slice and dynamic that will be fitted can be selected.

min R-square is the lowest R^2 accepted: where the fit of a pixel has a lower R^2 (0.80 by default), its T_2, M_0 and R^2 values are set to zero. So are T_2 values below 0 or above 5 s.

exclude noise (ticked by default) leaves out of each pixel's fit the echoes whose signal has dropped to the noise level, measured in the background around the mask. Echoes at the noise level make T_2 (and T_2*) come out too long where the signal is weak. Each pixel keeps at least its first three echoes, and one low echo does not count: the signal has to stay at the noise level. It needs a mask (step 4); without one nothing is left out. The message box says what share of the echoes was left out.

B_0 correction (**through-plane**), for multi-slice 2D multi-gradient echo data: tick it to correct T_2* for the signal that the field's change across the slice thickness takes away (through-plane dephasing). Without it, T_2* comes out too short where the field changes steeply, near air and bone. The field is measured from the echoes' phases. The correction needs the right TE0 and at least two slices; it does not apply to 3D scans or to the model-based fit, and the box is available only after the reconstruction of such a scan. The message box gives the field change across a slice, and how many pixels kept the uncorrected T_2* (where too little signal was left at the echoes to correct).

The bottom part of this tab lists the echoes. Only the echoes ticked are fitted, which can be used to leave out, for example, the first echo (left out by default for spin echo data), the last echoes below the noise level, or the even or odd echoes.

Press **Fit T_2** to start the T_2 fitting; **Abort** on the Dashboard stops it. The maps fill in slice by slice while the fit runs. Each pixel is fitted to the equation

> $Signal = M_0 \exp(-TE / T_2)$

For multi-gradient echo data the same equation gives T_2*.

Model-based fitting

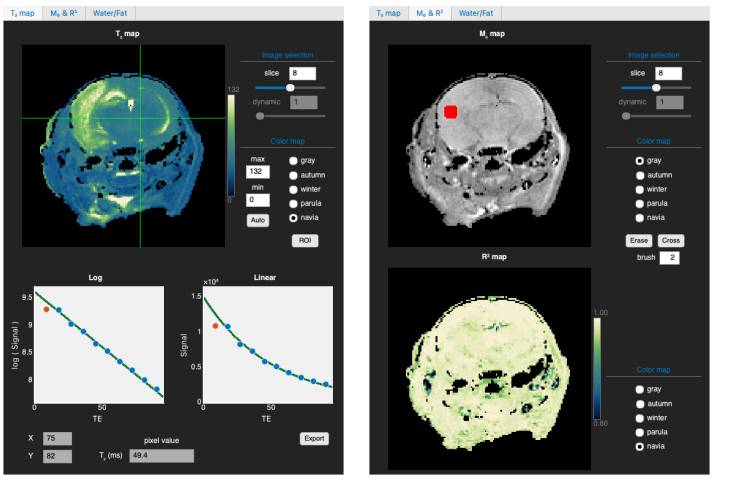
Tick **model-based** to fit a model-based reconstruction of spin echo data with BART: T_2 and M_0 are then fitted to the k-space directly. It gives no R^2 map and does not apply min R-square. It is offered for spin echo data when BART is available.

Step 6: Fit results

The T_2 maps are shown in the T_2 map tab, on the scale set with **max** and **min**, or with **Auto** (0 to twice the mean T_2 of the slice shown). An M_0 map and an R^2 map are shown in the second tab. A different color map can be chosen for each (gray, autumn, winter, parula, navia). The sliders beside the maps step through their slices and dynamics.

The color bars stand beside the T_2 and R^2 maps, with their end values above and below them: the scale's max and min for T_2, 1 and the R^2 cut-off for R^2.

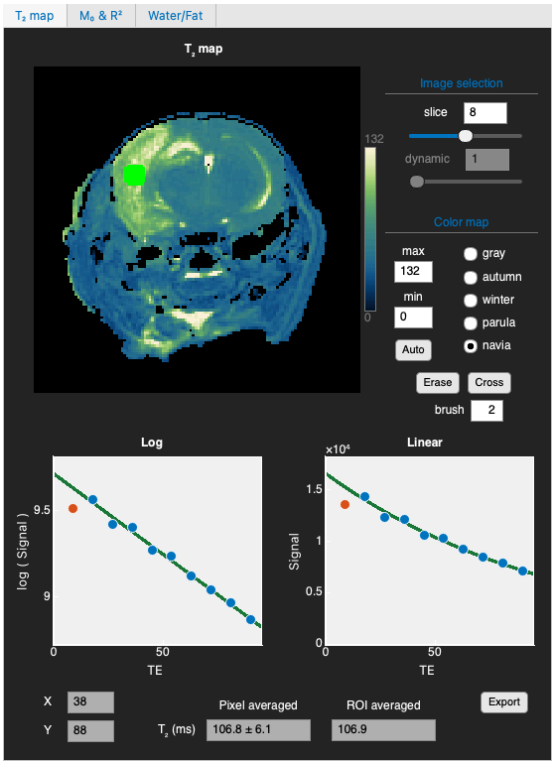
T_2 map **M_0 & R^2**



Step 7: T_2 values of pixels and ROIs

When you click on the T_2 map a cross-hair is shown, with the log(signal) and the signal of that pixel as a function of TE (symbols), the fitted curve (line) and its T_2 value (**Pixel value**). A click outside the mask shows nothing. A red symbol indicates that this echo was left out of the fit, according to the list in the Fit parameters tab (see step 5) — in the window on the cover, the first echo — or because its signal was at the noise level (**exclude noise**), or because the B_0 correction left too little of it. When the maps were fitted with the B_0 correction, the fit of the pixel or ROI has it too, and the curve shows the signal it takes away.

You can toggle between the cross-hair and region-of-interest (ROI) values with the **ROI** / **Cross** button. A single ROI can be 'painted' on the T_2 map, in one or more slices, by mouse clicks on the image, with the brush size in pixels set below the button.



For the ROI, two T_2 values are calculated:

- **Pixel averaged** — the average of the T_2 values of the pixels, with their spread: T_2(ROI) = 1 / mean(R_2 of the pixels).
- **ROI averaged** — the T_2 equation is fitted to the average signal intensities in the ROI, producing a single T_2 value for that ROI.

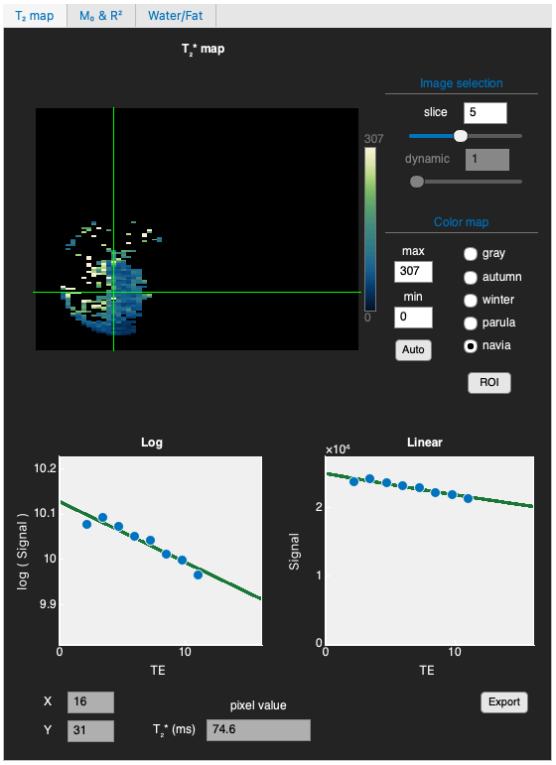
In the example, the lesion's T_2 (107 ms) is about twice that of the healthy brain beside it (49 ms, the cross-hair on the cover).

The ROI can be removed with **Erase**. The ROI-averaged signal is written to a CSV file with **Export**, `roiData-<tag>.csv`, beside the echo times, in a folder you choose.

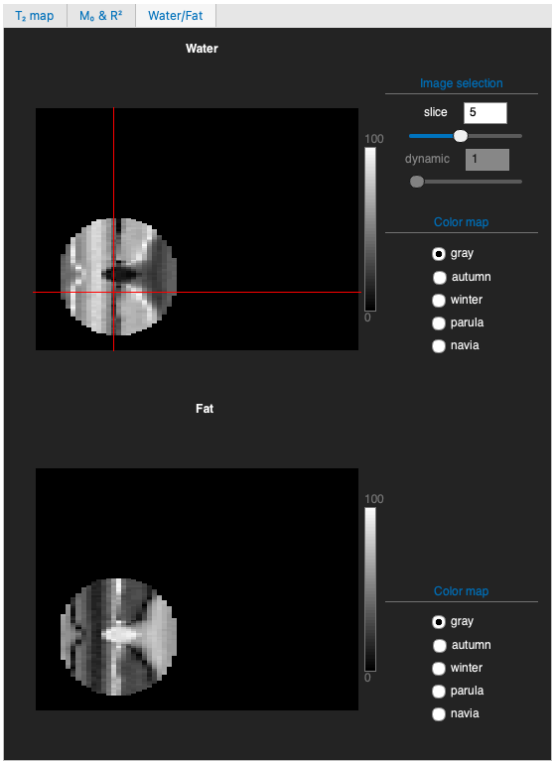
Step 8: T_2* and water/fat separation

Multi-gradient echo data are fitted in the same way; the maps, the fields and the plots then say T_2*.

The example below is a phantom whose echoes span only 11 ms, much shorter than its T_2*: the signal hardly decays, so the fit is uncertain. For this scan **min R-square** was lowered to 0.5, and still only part of the phantom passes.



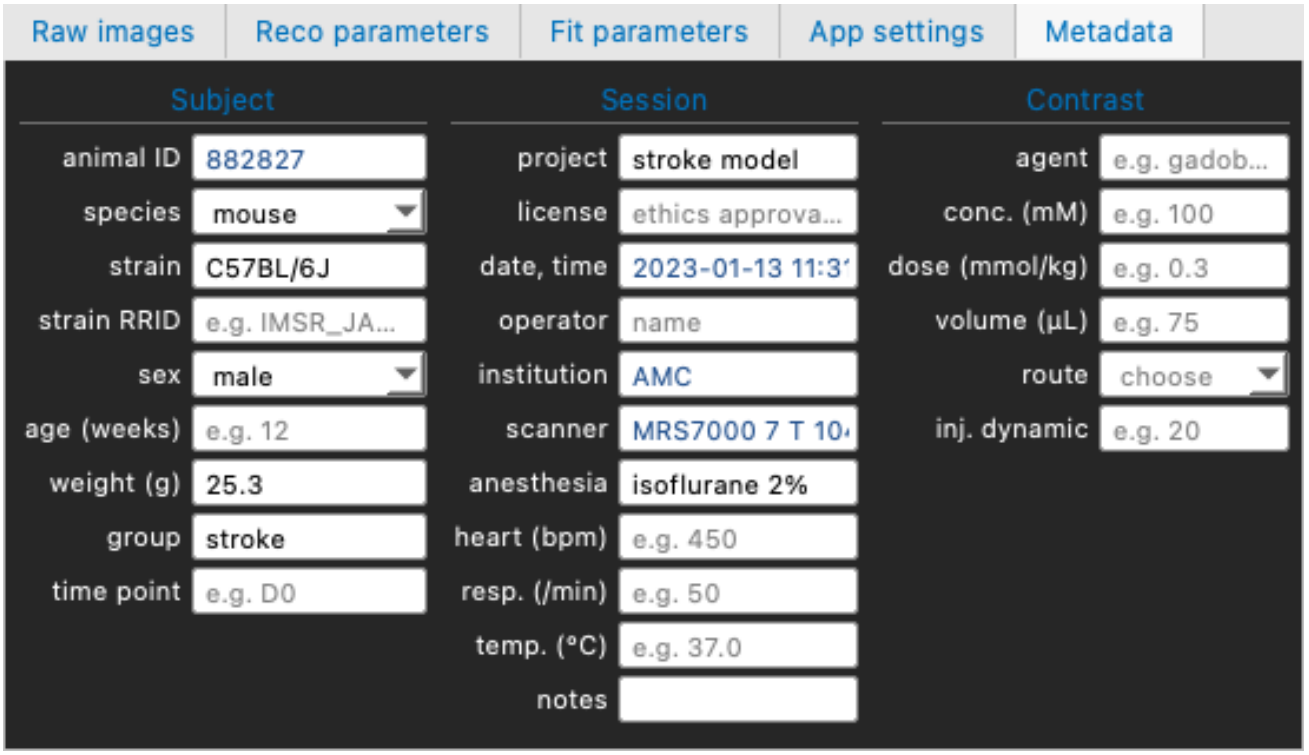
Water/fat. For multi-gradient echo data, tick **water/fat** on the Dashboard before pressing **Fit T_2** to also separate water from fat by their chemical shift. It needs at least 3 echoes. The water and fat maps, in percent of the total signal, are shown in the Water/Fat tab, each with its own color map and a color bar from 0 to 100 %.



Step 9: Metadata

The **Metadata** tab, beside the Raw images, Reco parameters, Fit parameters and App settings tabs, holds the record of the scan, which the DCM / NIFTI and GIF exports write with the maps. Its fields stand in three columns:

- **Subject:** animal ID, species, strain and its RRID, sex, age, weight, group and time point;
- **Session:** project, ethics license, date and time, operator, institution, scanner, anesthesia, heart and respiration rate, and temperature, with a field for notes;
- **Contrast:** agent, concentration, dose, volume, route and the dynamic it was injected at.



At a load the app fills in what the scan's files say: from the `.rpr` file beside the MRD file the animal ID and the time point (from the patient name typed at the console: `20240212_926752_D0` is animal 926752, day 0), the date and time, the institution and the scanner, and from the scanner's DICOM images, where they are found, the project. What the console leaves at its standard values (sex O, weight 0, the login as the operator) is left empty.

What was filled in for an earlier scan of the same exam is filled in again, from the records the apps wrote in the exam's `Metadata` folder (T2mapp, Retrospective, P2ROUD and the other mapping apps alike) and from the tab as it was for the scan loaded before: the animal's details are typed only once per exam. The date, the heart and respiration rate, the temperature, the notes and the injection's dynamic belong to one scan and are never taken from another.

A value the app filled in is shown in blue; every field can be corrected, and then counts as yours. Species, sex and route are lists to choose from, or to type in; numbers are kept with their units (a number field holding something else is edged in red). The species and sex are written with their codes (NCBI Taxonomy, PATO), so the record can be read by other software as well as by people.

Step 10: Exporting the maps

There are two ways to export the maps, and one for the ROI data.

1. **DCM / NIFTI** — exports the T_2, M_0 and R^2 maps, and the water and fat maps when they were made, in DICOM and NIfTI format for further processing in third-party software, and the T_2 map also as a BIDS quantitative map (`BIDS/derivatives/t2mapp/sub-<tag>/anat/sub-<tag>_T2map.nii.gz`, or `_T2starmap` for gradient echo data, in seconds; the DICOM and NIfTI maps are in ms). The program looks for the scanner's DICOM files of the scan and takes their header; if they are not found, the header, with the image position and orientation, is made from the MRD file, with the patient, institution and scanner of the scan's `.rpr` file (empty where it has none). To keep the decimals in the DICOM files, the T_2, R^2, water and fat maps are multiplied by

100, or by less where a map would not fit; the factor is in the header (Rescale Slope), so that viewers show the true values. The DICOM files carry the species, strain, sex, age and weight of the animal where the Metadata tab has them.

2. **GIF** — exports the maps as animated GIF files, one frame per slice, or one GIF per slice with a frame per dynamic, in the color maps and scales shown, with two pictures of the app's window.
3. **ROI data** — the **Export** button of the T_2 map tab (step 7).

When the MRD file lies in the scanner's folders, …`/MRD/<study>/<scan>`, the exports go to …`/Data/<study>/DICOM`, `Nifti`, `BIDS` and `GIF`, each into a new numbered folder; otherwise the app asks for a folder.

The DCM / NIFTI and GIF exports also write, in …`/Data/<study>/Metadata/<tag>T2/<n>` (or under the folder asked for), a new numbered folder each time:

- `metadata.json` — the Metadata tab's record;
- `t2mapp-report_<tag>.json` and `.txt` — the report of the export: the app and its libraries, the scan and a checksum of each of its files, its acquisition, every step of the session with its parameters, the reconstruction, registration and fit settings, the statistics of each map per slice and dynamic within the mask, the echo times and the echoes used, for the GIF export the scale and color maps, and every file the export wrote with its checksum;
- `ro-crate-metadata.json` — the same as an RO-Crate (https://www.researchobject.org/ro-crate/), a standard description of the folder for data repositories.

The paths in them are relative to the study folder (or the folder asked for), so they hold no names of your own folders. The ROI data export writes no record.

Step 11: Exit

Press **Exit** to shut down the program. Closing the window does the same, also while a step runs: the registration or fit is stopped, the window disappears, and the program closes as soon as the step has ended.

7. Credits

- **BART toolbox** — https://mrirecon.github.io/bart/. M. Uecker et al., Berkeley Advanced Reconstruction Toolbox, Proc. Intl. Soc. Mag. Reson. Med. 23, 2486 (2015).
- **sigpy** — Frank Ong and Michael Lustig, SigPy: a python package for high performance iterative reconstruction, Proc. Intl. Soc. Mag. Reson. Med. 27, 4819 (2019).
- **elastix** — S. Klein, M. Staring, K. Murphy, M.A. Viergever, J.P.W. Pluim, elastix: a toolbox for intensity based medical image registration, IEEE Transactions on Medical Imaging 29(1), 196–205 (2010). https://elastix.dev
- **PCA denoising** — J. Veraart, D. S. Novikov, D. Christiaens, B. Ades-Aron, J. Sijbers, E. Fieremans, Denoising of diffusion MRI using random matrix theory, NeuroImage 142, 394–406 (2016).
- **Gibbs ringing suppression** — Kellner, E., Dhital, B., Kiselev, V.G. and Reisert, M., Gibbs-ringing artifact removal based on local subvoxel-shifts. Magn Reson Med 76(5), 1574–1581 (2016).
- **Water/fat separation** — the ISMRM Fat-Water Toolbox: H.H. Hu, P. Börnert, D. Hernando et al., ISMRM workshop on fat-water separation: insights, applications and progress in MRI, Magn Reson Med 68(2), 378–388 (2012); its fw3pluspoint method: S.D. Sharma, H.H. Hu, K.S. Nayak, Accelerated water-fat imaging using restricted subspace field map estimation and compressed sensing, Magn Reson Med 67(3), 650–659 (2012).
- **Color maps** — F. Crameri, Scientific colour maps (navia), Zenodo, doi:10.5281/zenodo.1243862.
- **NIfTI conversion** — dcm2niix: C. Rorden et al., https://github.com/rordenlab/dcm2niix.